

System Documentation & Complete Walkthrough

Two complements in one volume: a precise technical reference for the system as currently built, and a from-scratch teaching guide that explains why every piece exists and how they cooperate.

PART I

Technical Reference

Stack, modules, RAG pipeline, security, infrastructure — precise facts, tables, versions.

PART II

Teaching Edition

Every component explained from scratch, two full request walkthroughs, layers, diagrams, glossary.

Basis

Onyx (OSS) core, reduced & rebranded

Date

July 19, 2026

§ Contents

PART I — TECHNICAL REFERENCE

I.01	Overview	What Orbyte is and its core components
I.02	System Architecture	Services and data flow
I.03	Backend	Stack, modules, auth, database, workers, search index
I.04	AI & Retrieval Pipeline	Chunking, embeddings, hybrid search, LLMs, agent tools
I.05	Data Connectors	Active sources and sync mechanics
I.06	Frontend	Stack, applications, admin dashboard
I.07	Security & Access Control	AuthN/AuthZ, permissions, encryption
I.08	Infrastructure & Deployment	Container services and topology
I.09	Technology Stack Reference	Consolidated version table

PART II — TEACHING EDITION

II.01	The Components, One at a Time	15 components, each explained from first principles
II.02	Story One — A Chat Question	Every hop, from click to streamed answer
II.03	Story Two — A PDF Upload	Every hop, from click to searchable document
II.04	The System in Layers	Eight layers and why each exists
II.05	The Full System, Visually	Complete architecture diagram, arrow by arrow
II.06	Quick-Reference Glossary	Every concept introduced, in one table

PART I

Technical Reference

Precise, current-state facts about the platform as it is actually built and wired today — stack versions, module inventory, the retrieval pipeline's exact parameters, and deployment topology. Written for someone who already knows the vocabulary and wants the numbers.

01 Overview

Product & component summary

Orbyte is an AI platform that lets users chat with an LLM grounded in their own connected data. It is built on a reduced, rebranded fork of the open-source Onyx platform: the full connector and admin surface of upstream Onyx has been trimmed to a focused core — chat, retrieval-augmented generation (RAG), a small set of first-party connectors, and access-controlled administration — while the underlying engine (indexing pipeline, hybrid search, multi-provider LLM routing, permissioning) remains fully intact.

Product surfaces

Surface	Route	Purpose
Chat	/app	Primary conversational interface: agents, streamed answers, citations, per-agent tools, Deep Research mode.
Craft	/craft	Sandboxed agent/app-building workspace — an in-product coding-agent environment with a live sandbox, file/diff/search tool cards, and approvals.
NRF	/nrf	Companion browser-extension surface: new-tab-page replacement and side-panel chat.
Admin	/admin	Configuration of LLMs, connectors, agents, users and groups.
Enterprise	/ee	Subscription-gated features: billing, whitelabel theming, usage analytics, query history.

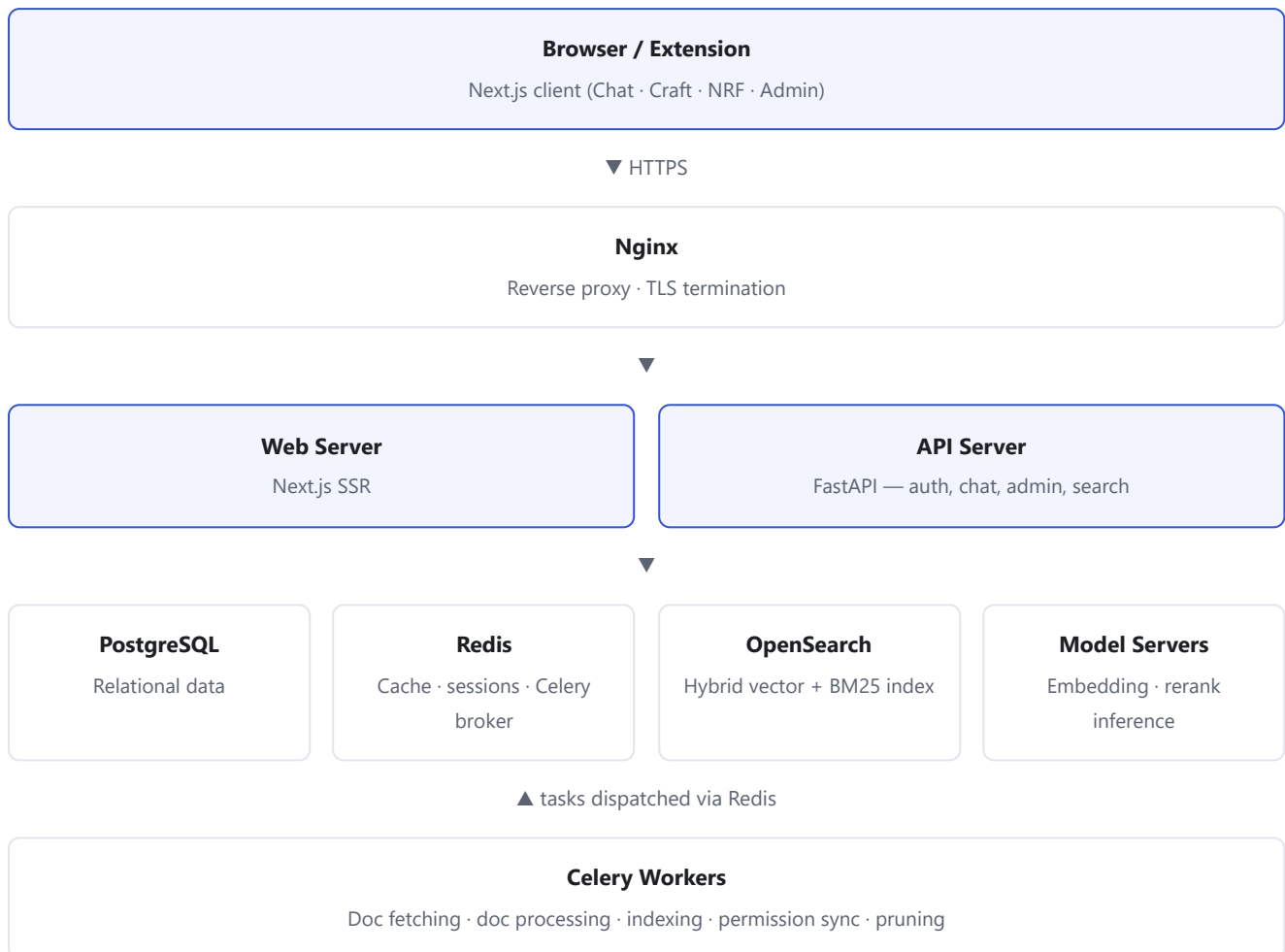
Core components at a glance

Component	Technology	Role
Web application	Next.js / React	All user-facing and admin UI
API server	FastAPI (Python)	Auth, chat, connector, admin, and search endpoints
Background workers	Celery + Redis	Connector sync, document processing, indexing, permission sync
Relational database	PostgreSQL	Users, permissions, connectors, chat history, metadata
Search index	OpenSearch	Hybrid vector + keyword (BM25) retrieval over document chunks
ML inference servers	SentenceTransformers / PyTorch	Embedding & reranking, isolated from the API server

Cache / broker	Redis	Sessions, query-embedding cache, Celery message broker
Object storage	MinIO (S3-compatible)	Uploaded files and generated artifacts
Sandbox	Docker-based code interpreter	Isolated code execution for the Python tool and Craft

02 System Architecture

Services & data flow



Indexing path

Celery Beat schedule → connector fetch → filter / dedup → chunk
→ embed (model server) → write to OpenSearch + PostgreSQL

Query path

Client request → API server (auth + permission check) → embed query
→ hybrid search in OpenSearch (permission-filtered) → optional rerank
→ prompt assembly → LLM (streamed) → citation mapping → SSE to client

Note. Permission filtering is applied *inside* the OpenSearch query itself (an `external_access` field on every document), not as a post-processing step — unauthorized chunks are never returned by the index.

3.1 — Core stack

Layer	Technology	Version
Language / runtime	Python	3.13
Web framework	FastAPI	0.133
ORM	SQLAlchemy	2.0.50
Migrations	Alembic	1.18
Validation	Pydantic	2.11
Task queue	Celery	5.5
LLM routing	LiteLLM	1.89
Search client	opensearch-py	3.2
ASGI server	Uvicorn	0.49
Embedding runtime	SentenceTransformers (PyTorch)	—

3.2 — Application modules

The backend is organized as a single Python package (`onyx/`) of focused modules. All listed modules below are actively wired into the running application (imported and registered from `main.py`), unless marked otherwise.

Module	Responsibility
<code>auth/</code>	Basic, Google OAuth, OIDC and SAML 2.0 authentication
<code>chat/</code>	Chat session/message handling, streaming response orchestration
<code>connectors/</code>	Source-system integrations (registry + sync interfaces)
<code>indexing/</code>	Chunking, contextual summarization, embedding orchestration
<code>document_index/</code>	Vector/keyword index abstraction (OpenSearch, Vespa)
<code>llm/</code>	Multi-provider LLM abstraction over LiteLLM
<code>tools/</code>	Agent tool registry and construction

<code>deep_research/</code>	Multi-step agentic research loop
<code>coding_agent/</code> · <code>sandbox_proxy/</code>	Craft coding-agent tool and its sandbox execution backend
<code>skills/</code>	Reusable AI skill definitions
<code>image_gen/</code>	Image generation tool + admin config
<code>voice/</code>	Speech-to-text / text-to-speech chat mode
<code>mcp_server/</code>	In-process Model Context Protocol tool client
<code>kg/</code>	Knowledge-graph admin configuration TOOL DISABLED
<code>background/</code>	Celery app, task definitions, worker entry points
<code>redis/</code>	Cache and distributed-lock abstractions
<code>file_store/</code>	Blob storage abstraction (MinIO / S3-compatible)
<code>db/</code>	SQLAlchemy models and queries (~154 model classes)
<code>server/</code>	All FastAPI route handlers
<code>prompts/</code>	System / user prompt templates
<code>tracing/</code>	OpenTelemetry request tracing
<code>ee/</code>	Enterprise-tier gated features (analytics, whitelabel, SCIM)

3.3 — Authentication & sessions

Four authentication backends are implemented and registered at startup: **Basic** (email + password, the default), **Google OAuth**, **OIDC**, and **SAML 2.0**. Sessions are stored server-side in Redis (not stateless JWT), default expiry 7 days, transported via an HTTP-only cookie — allowing instant session revocation. Role-based access control defines: Admin, Global Curator, Curator, Basic User, and Anonymous.

3.4 — Database

PostgreSQL is the system of record for everything except chunk vectors and full-text content. Schema evolution is managed through Alembic; the model layer (`db/models.py`) defines roughly 154 SQLAlchemy model classes. Central tables include users, connectors, credentials (encrypted), connector-credential pairs, documents, index attempts, chat sessions/messages, personas (AI agents), LLM provider configs, search settings, and document sets.

3.5 — Background processing

Celery, backed by Redis, runs dedicated worker pools per workload so each can be scaled independently:

Worker	Representative queues
Primary	<code>celery</code> — heartbeats, system checks
Doc-fetching	<code>connector_doc_fetching</code>
Doc-processing	<code>docprocessing</code> — chunk + embed + index
Heavy	<code>connector_pruning</code> , <code>connector_doc_permissions_sync</code> , <code>connector_external_group_sync</code>
Light	<code>vespa_metadata_sync</code> , <code>connector_deletion</code> , <code>doc_permissions_upsert</code>
User files	<code>user_file_processing</code> , <code>user_file_project_sync</code> , <code>user_file_delete</code>
Monitoring / Beat	<code>monitoring</code> , <code>scheduled_tasks</code> — periodic dispatch (cron-like)

3.6 — Search index

The system indexes into **OpenSearch 3.6** by default (`ENABLE_OPENSEARCH_INDEXING_FOR_ONYX=true`, `ONYX_DISABLE_VESPA=true`). The original Vespa index integration remains present in code but is disabled and not deployed — only an OpenSearch container is defined in the deployment stack. OpenSearch executes hybrid retrieval (dense vector similarity + BM25 keyword scoring) natively within a single query, with permission filters applied at the index layer.

4.1 — Document ingestion pipeline

1. Filter

– skip empty docs / docs exceeding max size

2. Vision pass

– LLM-generated text description for image sections

3. De-duplicate

– gate 1: updated_at unchanged → skip
gate 2: content-hash unchanged → skip

4. Upsert

– document metadata, tags, permissions → PostgreSQL

5. Chunk

– split into overlapping, sentence-aware chunks

6. Context (opt)

– LLM prepends a short context summary per chunk

7. Embed

– chunks → dense vectors via the embedding model server

8. Write

– chunks + vectors + text → OpenSearch

4.2 — Chunking

Parameter	Value	Notes
Default max chunk size	512 tokens	Primary retrieval unit
Chunk overlap	0	Chunks are split on sentence/section boundaries rather than relying on overlap
Minimum content threshold	256 tokens	Below this, adjacent content is merged into a chunk
Mini-chunk size	150 tokens	Secondary, finer-grained chunks used for multi-vector precision retrieval
Large-chunk ratio	4×	Optional coarser chunk grouping when multipass + large-chunk mode is enabled
Contextual RAG	OFF BY DEFAULT	When enabled, gpt-4o-mini generates a context blurb prepended to each chunk before embedding, improving retrieval for context-dependent passages

4.3 — Embedding model

Setting	Value
Default model	nomic-ai/nomic-embed-text-v1
Vector dimensions	768 (environment-overridable)

Normalization	Enabled
Serving	Dedicated model server processes (indexing & inference are separate containers), isolated from the API server for independent scaling
Legacy model	<code>thenlper/gte-small</code> (384-dim) — retained for migration compatibility

Reranking uses a cross-encoder model that is configured per deployment through Search Settings in the admin panel — there is no single hardcoded default; an administrator selects and activates a reranker model explicitly.

4.4 — Hybrid retrieval

Every query combines **dense vector similarity** (semantic match) with **BM25** keyword scoring (exact term / proper-noun match) in a single OpenSearch query, so the system catches both conceptual questions and literal identifiers (error codes, product names) that pure vector search tends to miss. Results are permission-filtered inside the same query, then optionally re-scored by the cross-encoder reranker before the top candidates are passed to the LLM.

4.5 — LLM providers

All LLM calls route through a single provider abstraction built on LiteLLM. Twelve providers have first-class configuration support in the admin UI:

`Anthropic` · `OpenAI` · `Azure OpenAI` · `AWS Bedrock` · `VertexAI` · `Ollama` · `LM Studio` · `OpenRouter` · `LiteLLM Proxy` · `Bifrost` · `Nebius` `TokenFactory` · `OpenAI-compatible endpoint`

4.6 — Agent tools

Tool	Status	Description
Search	ACTIVE	Core RAG retrieval over indexed documents
Web Search	REQUIRES PROVIDER KEY	Serper, Google PSE, Brave, SearXNG, Firecrawl or Exa
Open URL	ACTIVE	Fetch and read a specific web page
Python / Code Interpreter	ACTIVE	Sandboxed code execution (dedicated container)
Coding Agent (Craft)	FLAGGED OFF	Full agentic build/sandbox workspace; disabled by default (<code>ENABLE_CRAFT=false</code>)
File Reader	ACTIVE	Reads uploaded/attached files into context
Memory	OPT-IN	Per-user persistent memory, enabled per user account

Custom / OpenAPI Actions	BACKEND ONLY	Functional in the engine; its admin UI entry point is currently removed from navigation
MCP Tools	ACTIVE	In-process Model Context Protocol client; the standalone MCP server container is disabled by default
Image Generation	REQUIRES CONFIG	Active only once an image-generation model is configured
Deep Research	ACTIVE	Not a tool call — a distinct multi-step agentic chat mode (plan → search → synthesize, iterated)
Knowledge Graph	DISABLED	Registered but its construction path is disabled; not reachable from any persona

4.7 — Chat request lifecycle

1. POST /chat/send-message (SSE connection opened)
2. Session validated against Redis-backed cookie
3. Persona + document-set scope resolved; user group memberships loaded
4. Query embedded (query-embedding cache in Redis, 15 min TTL)
5. Hybrid search executed in OpenSearch, permission- and scope-filtered
6. Optional cross-encoder reranking of top candidates
7. Prompt assembled: persona system prompt + history + retrieved chunks
8. LLM invoked via LiteLLM; tokens streamed back over SSE
9. Citation markers parsed and mapped to source documents
10. Assistant message + citations persisted to PostgreSQL

05 Data Connectors

Active sources & sync

The connector engine supports 54 source integrations in the backend registry. The current product intentionally exposes only five in the Add-Connector UI, keeping the surface area small and predictable:

Connector	Syncs	Auth
File Upload	Manually uploaded documents (PDF, Word, etc.)	—
Web Crawler	Public web pages from a seed URL	—
Google Drive	Docs, Sheets, Slides, and files in a Drive	OAuth 2.0
Confluence	Spaces and pages	API token / OAuth
Slack	Channels and message history	OAuth / bot token

Note. The remaining 49 connectors (Jira, Notion, GitHub, Zendesk, Salesforce, Gmail, S3, and others) remain fully implemented server-side and require no backend changes to re-enable — only removing them from the UI allowlist that currently restricts the Add-Connector page.

Sync mechanics

```
Celery Beat → connector due for sync? → doc-fetching queue
→ connector instantiated, documents pulled (full or incremental)
→ batches dispatched to doc-processing queue → chunk / embed / index
→ sync status recorded in PostgreSQL (IndexAttempt)
```

Most connectors support time-windowed incremental polling (only documents changed since the last successful sync); complex sources use a checkpointed, resumable sync. A separate permission-sync path mirrors source-system access-control lists into Orbyte so retrieval stays permission-aware.

6.1 — Core stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.2
UI library	React	19.2
Language	TypeScript	5.9
Styling	Tailwind CSS	4.3
State management	Zustand	5.0
Data fetching	SWR	2.1
Forms & validation	Formik + Yup	—
Component primitives	Radix UI	—
Design system	Internal package <code>@onyx-ai/opal</code>	—
Package manager / runtime	Bun	—
Lint / format	oxlint / oxfmt	—
Testing	Jest, Testing Library, Playwright, Storybook	—

6.2 — Application surfaces

Chat (`/app`)

Primary conversational UI. Streams answers token-by-token over Server-Sent Events, renders inline citations, and exposes per-agent tools dynamically depending on which tools that agent is configured with. A Deep Research toggle is always available as a top-level chat mode.

Craft (`/craft`)

A sandboxed agent-building workspace — bash, diff, search, and web-fetch tool cards, a live sandbox preview, tool-call approvals, and sub-agents. Effectively an in-product coding-agent environment

NRF (`/nrf`)

The companion browser extension's surface: a new-tab-page replacement and a side-panel chat, offering a compact agent selector, model picker, and quick access into the full chat app.

Admin (`/admin`)

Configuration dashboard for LLMs, connectors, agents, and users — see 6.3.

for building custom apps. Gated off by default at the backend (`ENABLE_CRAFT=false`).

6.3 — Admin dashboard navigation

The navigation currently exposes:

- **Configuration** — LLM Models, Web Search, Chat Preferences
- **Agents & Actions** — Agents
- **Documents & Knowledge** — Indexing Status, Add Connector, Document Sets, Index Settings
- **Permissions** — Users, Groups

The following admin pages exist in code but have been deliberately removed from the navigation menu (reachable only by direct URL, if authorized): **Service Accounts**, **Query History**, **OpenAPI Actions**, **SSO Providers**, **SCIM**, and **Hooks/Webhooks**. This keeps the admin surface minimal without discarding the underlying functionality.

6.4 — Enterprise-tier UI

Billing, whitelabel theming, custom analytics, and query history live under `/ee` , gated by subscription tier. Below the required tier, matching entries render as disabled with an upgrade prompt rather than being hidden outright.

07 Security & Access Control

AuthN/AuthZ, permissions, encryption

7.1 — Authentication

- Passwords hashed via `fastapi-users` (bcrypt); configurable complexity policy (min/max length, character requirements)
- Sessions held server-side in Redis, default 7-day expiry, HTTP-only cookies (not readable by client-side JavaScript)
- Google OAuth, OIDC, and SAML 2.0 available alongside Basic auth for enterprise identity federation

7.2 — Authorization

Role-based access control governs administrative capability (Admin, Global Curator, Curator, Basic User, Anonymous). Independently, every indexed document carries an access-control field synchronized from its source system; retrieval queries are filtered by this field at the index layer, so a document a user cannot access is never returned — regardless of semantic relevance.

7.3 — Credential & data protection

- Connector credentials (OAuth tokens, API keys) encrypted at rest using a dedicated encryption key, independent of database access
- Credentials displayed in the admin UI are masked by default
- All secrets are supplied via environment variables, never hardcoded

The system is deployed as a set of Docker Compose services:

Service	Role	Status
api_server	FastAPI backend	ACTIVE
web_server	Next.js frontend	ACTIVE
background	Celery workers (all queues)	ACTIVE
relational_db	PostgreSQL 15.2	ACTIVE
opensearch	Search index, v3.6	ACTIVE
cache	Redis	ACTIVE
inference_model_server	Query-time embedding / rerank inference	ACTIVE
indexing_model_server	Indexing-time embedding inference	ACTIVE
minio	Object storage	ACTIVE
code-interpreter	Sandbox for code execution tools	ACTIVE
ollama	Local LLM runtime option	ACTIVE
nginx	Reverse proxy, TLS termination	ACTIVE
mcp_server	Standalone MCP server process	DISABLED

The ML inference workload is deliberately split from the API server into its own containers, so embedding/reranking throughput can be scaled (including onto GPU hardware) independently of API request handling. Kubernetes/Helm and Terraform deployment paths are retained alongside the Docker Compose stack for production environments.

09 Technology Stack Reference

Consolidated version table

Category	Technology	Version / Notes
Backend language	Python	3.13
Web framework	FastAPI	0.133
ORM / migrations	SQLAlchemy / Alembic	2.0.50 / 1.18
Validation	Pydantic	2.11
Task queue	Celery	5.5
LLM routing	LiteLLM	1.89
Relational database	PostgreSQL	15.2
Search index	OpenSearch	3.6
Cache / broker	Redis	5.0 client
Object storage	MinIO	S3-compatible
Embedding model	nomic-ai/nomic-embed-text-v1	768-dim
Embedding runtime	SentenceTransformers / PyTorch	—
Frontend framework	Next.js	16.2
UI library	React	19.2
Language	TypeScript	5.9
Styling	Tailwind CSS	4.3
Containerization	Docker / Docker Compose	—
Orchestration (production)	Kubernetes + Helm, Terraform	—
Reverse proxy	Nginx	—

PART II

Teaching Edition

A from-scratch explanation of the same system, written for someone who knows how to program but has never built a production AI platform. Every component is explained in plain English: what it is, why it exists, who talks to it, and what breaks without it. Then we trace two complete user actions — a chat question and a document upload — hop by hop, followed by a layered view and full system diagrams.

II.01 The Components, One at a Time

15 building blocks, explained from first principles

For every component below we answer the same eleven questions: what it is, why it exists, when it activates, who it talks to, what data enters it, what it does internally, what it outputs, where that output goes, why this technology was chosen over an alternative, what breaks if it stops, and a real-world analogy. New concepts are called out the first time they appear.

1 The Browser (the client)

- 1. What is it?** The program on the user's own computer — Chrome, Edge, Safari — that renders the Orbyte page and captures clicks and keystrokes. It runs on the user's machine, not ours.
- 2. Why does it exist?** Something has to display pixels to a human and capture their input. Without it, nothing downstream — servers, databases, AI models — has any way to receive a question or show an answer.
- 3. When does it become active?** The instant the user opens the URL. From that moment it is alive and issuing requests.
- 4. What communicates with it?** Only Nginx. The browser never talks directly to FastAPI, PostgreSQL, or anything else.
- 5. What data enters it?** First an HTML/JS/CSS bundle; later, a steady trickle of small text fragments as an AI answer streams in.
- 6. What processing does it perform?** It runs the downloaded JavaScript (React), tracks what's typed, decides when to fire a request, and appends streamed text fragments to the screen as they arrive — which is why an answer appears to "type itself."
- 7. What does it output?** Outgoing HTTP requests — "log this user in," "here is the message the user just sent."
- 8. Where does that output go?** To Nginx — never directly to the API.
- 9. Why this technology?** Not a real "choice" — every interactive web app requires some browser. The actual design decision is what the browser is asked to run (a Next.js/React app — see Component 3).
- 10. What if it stopped working?** Nothing server-side would notice — but no human could interact with Orbyte at all. Its failure is invisible to the servers but total for the user.

REAL-WORLD ANALOGY

The browser is the customer at the restaurant counter. The kitchen, the fridge, the supplier trucks mean nothing if there's no counter where a person can place an order.

2 Nginx (the reverse proxy)

CONCEPT — REVERSE PROXY

A normal proxy hides a client from the internet. A reverse proxy hides the servers instead — the outside world sees one address (Nginx), which privately forwards each request to whichever internal server should handle it. The browser never even knows FastAPI and Next.js are separate machines.

- 1. What is it?** A fast, lightweight server whose only job is to sit at the front door of the system and decide where each incoming request goes.
- 2. Why does it exist?** Security — only Nginx is exposed to the internet, so it's the only thing that needs hardening — and traffic direction — a request to `/api/...` must reach FastAPI, a request for `/app` must reach Next.js.
- 3. When does it become active?** On every single request, before anything else server-side.
- 4. What communicates with it?** Upstream: the browser. Downstream: the Next.js web server and the FastAPI API server.
- 5. What data enters it?** Raw encrypted HTTPS traffic — method, path, headers (including the session cookie), and body.
- 6. What processing does it perform?** It terminates TLS (decrypts HTTPS back to plain HTTP) and forwards based on the URL pattern — it never inspects the meaning of the data.

CONCEPT — TLS / HTTPS

TLS is the encryption layer scrambling traffic between browser and server so nobody snooping on the network can read it. Nginx holds the certificate and does this decryption, so internal servers never deal with certificates directly.

- 7. What does it output?** The same request, forwarded to an internal address like `web_server:3000` or `api_server:8080`.
- 8. Where does that output go?** To Next.js or to FastAPI, depending on the path.
- 9. Why this technology?** Over alternatives like Apache/Caddy, Nginx handles very many simultaneous open connections with little memory — important for streaming chat connections — and it's the industry-standard choice, meaning it's extremely well understood and battle-tested.
- 10. What if it stopped working?** Total outage — nothing reaches any other server, no matter how healthy they are.

REAL-WORLD ANALOGY

The receptionist at a large office building — visitors don't wander the halls; the receptionist points them to the right floor, without doing the department's actual work.

3 Next.js (the web/frontend server)

- 1. What is it?** A framework built on React with its own small server, responsible for assembling and delivering the HTML/CSS/JavaScript that becomes the Orbyte website in your browser.
- 2. Why does it exist?** A real app has dozens of pages sharing components (buttons, sidebars) and needing consistent behavior (routing without a full reload, data fetching, loading states). Next.js supplies this scaffolding so the team builds pages and components, not plumbing.
- 3. When does it become active?** When Nginx forwards it a page-route request (e.g. `/app` , `/admin` , `/auth/login`).
- 4. What communicates with it?** Upstream: Nginx. It does not talk to PostgreSQL, Redis, or AI models directly — the JavaScript it ships to the browser later calls FastAPI directly (via Nginx) for data.
- 5. What data enters it?** HTTP requests for page routes, plus a session cookie if logged in.
- 6. What processing does it perform?** Assembles the requested page's HTML skeleton and bundles only the JS/CSS needed for that route.
- 7. What does it output?** HTML + JavaScript + CSS.
- 8. Where does that output go?** Back through Nginx to the browser, whose JavaScript then takes over and calls the API directly.
- 9. Why this technology?** React has the largest ecosystem, hiring pool, and tooling of the major UI frameworks; Next.js adds server-side rendering and file-based routing, reducing boilerplate versus hand-built routing.
- 10. What if it stopped working?** No page could load at all — a blank screen — even if every other server were perfectly healthy.

REAL-WORLD ANALOGY

The print shop that assembles a restaurant's menu before it reaches the customer — deciding layout and sections — while the actual food (data) still comes from the kitchen (FastAPI) later.

4 FastAPI (the API server — "the brain")

1. What is it? The Python web framework running the central application server that holds almost all business logic: authentication, permissions, chat orchestration, connector management, and the AI pipeline's entry points.

2. Why does it exist? The browser can't be trusted to enforce rules — a user could tamper with client-side JavaScript. Every decision that matters ("is this password correct," "can this user see this document") must run on a server the user cannot alter.

3. When does it become active? Whenever Nginx forwards a request whose path starts with something like `/api/...`.

4. What communicates with it? Upstream: Nginx. Downstream: PostgreSQL, Redis, OpenSearch, the embedding/reranking model servers, LiteLLM, Celery, and MinIO.

5. What data enters it? Structured JSON bodies, an auth cookie, and raw file bytes for uploads.

6. What processing does it perform? The busiest component in the system — on one request it may verify a session, load permissions, check the embedding cache, call the embedding model, query OpenSearch with a permission filter, call the reranker, assemble a prompt, call the LLM, and persist the result.

CONCEPT — SSE (SERVER-SENT EVENTS)

A normal request-response is one letter, one reply. SSE keeps the connection open and sends many small messages over time as they're ready — this is how an AI answer appears to type itself instead of arriving all at once. It's one-directional (server → browser), simpler than a two-way connection, because answers only need to flow one way.

7. What does it output? JSON for simple requests, or a continuous SSE stream of text fragments for chat answers.

8. Where does that output go? Back through Nginx to the browser, and, for many requests, into PostgreSQL (persisting results) and/or Redis/Celery (scheduling background work).

9. Why this technology? Over Flask/Django: FastAPI's `async / await` model lets one server handle many concurrent requests efficiently while waiting on slow calls (LLM, database); it auto-validates request/response shapes via Pydantic; and it auto-generates interactive API docs — a large win across 50+ endpoints.

10. What if it stopped working? Total functional outage — the page might still load, but no login, chat, or configuration action would work.

REAL-WORLD ANALOGY

The head chef in the kitchen: every order passes through them; they validate it, pull ingredients (PostgreSQL/Redis), delegate to specialists (Celery, the embedding model, the LLM), and plate the final dish.

5 PostgreSQL (the relational database)

CONCEPT — RELATIONAL DATABASE

Imagine spreadsheets that reference each other by ID (a Users sheet; a ChatSessions sheet where each row points back to a Users row). A relational database stores, queries, and enforces consistency across many such interlinked "spreadsheets" reliably, even with thousands of simultaneous readers/writers.

1. What is it? A relational database — data stored durably in tables of rows and columns, with precise, reliable answers even after a crash.

2. Why does it exist? The system needs one trustworthy source of truth for things that must never be lost or become inconsistent: passwords, connectors, chat history, permissions.

CONCEPT — ACID / TRANSACTIONS

A transaction is a group of changes that either all happen or none do. "Create a chat session AND insert its first message" should never leave a session with zero messages because the server crashed mid-way. PostgreSQL guarantees this.

3. When does it become active? Whenever FastAPI or a Celery worker needs to read or write a structured fact.

4. What communicates with it? Only FastAPI and Celery workers — never the browser, Nginx, or Next.js directly.

5. What data enters it? User accounts, hashed passwords, connector configs, encrypted credentials, chat sessions/messages, persona definitions, document metadata, and permission records.

6. What processing does it perform? Enforces structure and relationships, and answers queries efficiently using indexes.

7. What does it output? Rows of structured query results.

8. Where does that output go? Back into FastAPI's logic, then shaped into a JSON response — or used internally to decide what to do next.

9. Why this technology? Over MySQL: mature, open-source, rich support for complex queries and JSON columns, clean SQLAlchemy integration. It is deliberately not used for the AI search index — a different workload entirely (see OpenSearch).

10. What if it stopped working? Nobody could log in or load anything, even if the AI pipeline were perfectly healthy — the system has no memory of who anyone is without it.

REAL-WORLD ANALOGY

A hospital's official patient records office — every prescription and appointment is filed precisely and never lost. A doctor (FastAPI) consults it constantly, but it doesn't diagnose anyone itself.

6 Redis (cache + message broker)

- 1. What is it?** An extremely fast, in-memory key-value store — data lives in RAM, roughly 100-1000× faster than disk-based PostgreSQL, at the cost of being less durable.
- 2. Why does it exist?** Two jobs: (A) *Cache* — avoid redoing expensive, repeated computations; (B) *Message broker* — hand off work that shouldn't happen inside the live request, to run later in the background.
- 3. When does it become active?** On every request (session lookup); right before embedding a chat question (cache check); and whenever FastAPI hands off background work.
- 4. What communicates with it?** FastAPI reads/writes it constantly; Celery workers read from it (to pick up jobs) and write to it (status).
- 5. What data enters it?** Session tokens mapped to user IDs, temporarily-cached embedding vectors, and small JSON task messages.
- 6. What processing does it performs?** Almost none — it's a dictionary. The one behavior it enforces is expiration.

CONCEPT — TTL (TIME TO LIVE)

When you store something in Redis you can say "forget this after 15 minutes." A cache is only correct as long as reality hasn't changed underneath it; a short TTL keeps caching fast most of the time while staying self-correcting.

- 7. What does it output?** The cached value, or a "not found" (cache miss) telling the caller to compute it themselves.
- 8. Where does that output go?** Back into FastAPI's logic — a hit skips a slow step entirely; a miss triggers the slow work, whose result is then written back into Redis.

WORKED EXAMPLE

A user asks "What is our company's vacation policy?" FastAPI is about to embed this sentence. Before calling the (relatively slow, GPU-using) embedding model, it asks Redis: "Have you seen this exact text in the last 15 minutes?" If yes, Redis instantly returns the previously computed vector and the embedding model call is skipped entirely. If no, FastAPI calls the embedding model, stores the fresh vector in Redis for 15 minutes, and continues. Without Redis, ten employees asking a common question within the same few minutes would cause the embedding model to redundantly recompute the identical vector ten times.

- 9. Why this technology?** Over PostgreSQL: PostgreSQL is optimized for durability and complex relationships, which makes it inherently slower per operation — needless overhead for something as disposable as a 15-minute cache. Over a plain Python dictionary: FastAPI runs as multiple separate processes/containers, so an in-process dictionary in one container is invisible to another handling the next request — Redis is shared, external memory all instances can see.

- 10. What if it stopped working?** Every user would be logged out immediately (sessions live in Redis), embeddings would be recomputed every time, and Celery workers would stop receiving new jobs entirely.

REAL-WORLD ANALOGY

The short-term memory of a hospital triage nurse — sticky notes: "Room 4 already got vitals 10 minutes ago." Fast and deliberately temporary; the permanent record still lives in the records office (PostgreSQL).

7 Celery (background workers)

CONCEPT — WORKER & TASK QUEUE

A "task" is a description of work to do. A "queue" is an ordered waiting line of tasks. A "worker" is a process that watches the queue, grabs the next task, executes it, and repeats. Many workers can pull from the same queue at once, so work happens in parallel.

- 1. What is it?** A Python library for running tasks in the background, independent of the request that triggered them, working together with Redis as the delivery mechanism.
- 2. Why does it exist?** Some jobs — extracting text from a 200-page PDF, syncing an entire Drive — take too long to run inside a live HTTP request; doing so would freeze the user's browser and tie up a server that other users need.
- 3. When does it become active?** The instant FastAPI (or the scheduled "Celery Beat" clock) drops a task message into a Redis queue; an always-running worker notices within a fraction of a second.
- 4. What communicates with it?** Redis delivers tasks and receives status. Once running, a worker talks to PostgreSQL, connector code, the chunker, the embedding model, and OpenSearch.
- 5. What data enters it?** A small task description (e.g. `{"task": "process_document", "document_id": 4821}`) — a pointer, not the heavy data itself.
- 6. What processing does it perform?** Whatever the task requires — different worker types specialize in different queues (e.g. `docprocessing` for chunk/embed, `heavy` for slow permission syncs) so one kind of job can't starve another.
- 7. What does it output?** Side effects: rows written to PostgreSQL, chunks written to OpenSearch.
- 8. Where does that output go?** Into PostgreSQL and OpenSearch, ready for a future chat query to find — possibly hours after the original upload request finished.
- 9. Why this technology?** Over plain Python threads: threads share one process — a crash loses every in-progress task with no record, and threads can't scale across machines. Celery tasks are durable messages in Redis, retryable, and can run across 20 worker processes on several machines. Over hand-rolling a queue: Celery is mature, with built-in retries, scheduling (Beat acts like cron), and monitoring.
- 10. What if it stopped working?** Already-indexed data would remain searchable, but no new document would ever be processed, and permission changes would never propagate — the system would slowly go stale.

REAL-WORLD ANALOGY

Prep cooks working in the back, separate from the line cooks serving live orders. A slow-cooked item's ticket (Redis) is handed off and picked up by a free prep cook (a worker) whenever ready, adding to inventory (PostgreSQL/OpenSearch) for whenever it's needed.

CONCEPT — EMBEDDING

Imagine plotting every sentence as a point in space, positioned so similar-meaning sentences land near each other, even with completely different words. An embedding model is a neural network trained to compute those coordinates — here, 768 numbers per piece of text (a "vector"), instead of the 2 or 3 you could easily picture.

- 1. What is it?** A machine learning model — `nomiic-embed-text-v1` — running on its own server, converting text into a 768-number vector capturing its meaning.
- 2. Why does it exist?** Computers can't natively judge "similar meaning" — only exact characters. Keyword-only search misses obvious cases ("PTO policy" should find "Vacation Guidelines"). Embeddings turn that fuzzy notion into a precise, comparable distance in space.
- 3. When does it become active?** At indexing time (every chunk gets embedded once) and at query time (every non-cached user question gets embedded).
- 4. What communicates with it?** FastAPI (live queries) and Celery workers (bulk indexing) call it; it returns to whichever called it.
- 5. What data enters it?** Plain text — a question, or a ~512-token document chunk.

CONCEPT — TOKEN

Roughly a word or word-fragment — the unit an AI model processes internally. "512 tokens" is roughly 350-400 English words.

- 6. What processing does it perform?** Runs the text through a pre-trained neural network (PyTorch, via SentenceTransformers), outputting a fixed-length, normalized 768-number vector.
- 7. What does it output?** A vector, e.g. `[0.021, -0.183, 0.442, ...]`.
- 8. Where does that output go?** At indexing time, into OpenSearch attached to that chunk; at query time, into an OpenSearch search request and a 15-minute Redis cache entry.
- 9. Why this technology?** Runs on a separate server from FastAPI because embedding is computationally heavy and benefits from independent scaling (e.g. onto GPUs) without risking the login/chat server. `nomiic-embed-text-v1` specifically is open-source and can run entirely on infrastructure you control, so sensitive company documents never have to leave the deployment just to be embedded.
- 10. What if it stopped working?** No new document could be indexed, and no new question could be searched — cached recent questions would still resolve via Redis for 15 minutes, then everything halts.

REAL-WORLD ANALOGY

A librarian who shelves books by meaning rather than title, placing "medieval history" books together regardless of title wording — and who "reads" your question and walks you straight to the right shelf.

9 OpenSearch (the search index)

- 1. What is it?** A database purpose-built for search — storing every chunk's text and embedding vector, and ranking/returning the most relevant ones out of millions, fast.
- 2. Why does it exist?** PostgreSQL is great at "give me row 4821," not at "which of 2 million chunks is most semantically similar to this vector, restricted to what this user may see, ranked by relevance?" — a fundamentally different problem needing purpose-built data structures.
- 3. When does it become active?** When a worker finishes chunking+embedding a document (write), and when a live query needs content (read).
- 4. What communicates with it?** Celery workers write; FastAPI reads (searches) during chat.
- 5. What data enters it?** At write: chunk text, its vector, and metadata including `external_access` — exactly who is allowed to see it. At read: a query vector, raw query text, and the requesting user's identity/groups.
- 6. What processing does it perform?** Hybrid search:

CONCEPT — VECTOR SEARCH VS. BM25 VS. HYBRID

Vector search compares your question's embedding against every stored chunk's vector and finds the closest — great for conceptual questions even with different wording. BM25 is a decades-old statistical keyword-matching algorithm — great for exact terms, error codes, and proper nouns. Hybrid search runs both and merges the results, catching whichever a single method alone would miss.

OpenSearch also applies the permission filter during this same query — a chunk the user isn't authorized to see is never returned, not fetched-then-hidden.

- 7. What does it output?** A ranked, permission-filtered list of the top ~50 matching chunks.
- 8. Where does that output go?** To the reranker, and ultimately into the LLM prompt.
- 9. Why this technology?** Over PostgreSQL extensions: purpose-built for this exact workload at scale — millions of vectors, high throughput, native BM25, hybrid-query syntax. (The system originally used Vespa; the default has since moved to OpenSearch for its more standard operational tooling and managed-hosting options — Vespa's code path remains but is disabled.)
- 10. What if it stopped working?** No chat query could retrieve any document content — the LLM would answer, if at all, from generic training knowledge only, with no permission enforcement.

REAL-WORLD ANALOGY

A giant library's card catalog crossed with a security desk — pointing you to every relevant book by meaning and keyword, while checking your badge and only showing shelves you're cleared for.

10 The Reranker (cross-encoder)

CONCEPT — BI-ENCODER VS. CROSS-ENCODER

The embedding model is a "bi-encoder" — it looks at your question alone and each chunk alone, independently, ahead of time — fast enough for millions of comparisons. A cross-encoder reads the question and one specific chunk together, catching subtle relationships a bi-encoder misses — far more accurate, but too slow for millions of chunks. So the system uses the cheap bi-encoder to narrow millions down to ~50, then the expensive cross-encoder to carefully re-judge just those 50.

1-2. What is it, and why? A second model that re-scores OpenSearch's ~50 candidates more carefully, one (question, chunk) pair at a time, to correct the limits of the fast first-pass ranking before the most context-limited, expensive step — the LLM call.

3. When does it activate? Immediately after OpenSearch returns its candidates, before the prompt is built.

4-5. Communication & input. FastAPI sends it (question, chunk) pairs for each of the ~50 candidates.

6-7. Processing & output. Outputs a relevance score per pair; FastAPI re-sorts and keeps the top 10-15.

8. Where does the output go? Into prompt-building, the final step before calling the LLM.

9. Why two models instead of one? Cross-encoders alone are too slow at full scale; bi-encoders alone are fast but less precise on close calls. Sequencing "cheap and broad, then expensive and narrow" gets both speed and accuracy. Unlike the embedding model, no reranker is hardcoded — an administrator configures which cross-encoder to use, since organizations trade latency for precision differently.

10. What if it stopped working? Search still functions on OpenSearch's original ranking alone — answer quality may degrade slightly on ambiguous cases.

REAL-WORLD ANALOGY

If the embedding model is the librarian pointing you to the right section, the reranker is a specialist who personally reads all 50 shortlisted books and hands back only the 10 that truly answer your question, in the best order.

11 LiteLLM (the LLM router)

- 1. What is it?** A Python library providing one consistent way to call any LLM provider — OpenAI, Anthropic, Google, a local model — despite each having a completely different underlying API.
- 2. Why does it exist?** Without it, supporting a new provider would mean writing brand-new code for that provider's format every time. LiteLLM absorbs that variation so the rest of the codebase is written once and works against every provider.
- 3. When does it activate?** The moment FastAPI has assembled the final prompt and is ready to generate an answer.
- 4. What communicates with it?** FastAPI calls it; it calls out to whichever LLM provider is configured.
- 5. What data enters it?** The assembled prompt (system instructions, history, retrieved chunks, question) and provider configuration.
- 6. What processing does it perform?** Translates the generic request into the provider's exact format, and translates the streaming response back into a consistent internal format.
- 7. What does it output?** A stream of small text fragments as the LLM generates them.
- 8. Where does that output go?** Back to FastAPI, forwarded onward over SSE to the browser, and accumulated for saving to PostgreSQL once complete.
- 9. Why this technology?** Over integrating directly with one provider's SDK: being locked to one provider means being at the mercy of its pricing, uptime, and policy, and makes provider choice (e.g. an enterprise requiring Anthropic, or a fully self-hosted model for data sovereignty) impossible. LiteLLM is what makes 12-provider support realistic instead of 12× the engineering work.
- 10. What if it stopped working?** No chat answer could be generated — retrieval would still work, but nothing could write the response in natural language.

REAL-WORLD ANALOGY

A universal power adapter for international travel — the wall socket differs per country, but the adapter presents one consistent plug on your side.

12 The Chunker (document splitting)

- 1. What is it?** Code that splits a full document — one paragraph or five hundred pages — into smaller, roughly-uniform "chunks" before anything gets embedded.
- 2. Why does it exist?** An embedding model can't meaningfully compress an entire 200-page manual into one vector without blurring its meaning; and an LLM's limited context window means only a handful of focused excerpts can be handed over, not whole documents.
- 3. When does it activate?** During indexing, right after a document is fetched and passes initial checks.
- 4. What communicates with it?** A Celery worker calls it with raw extracted text; it hands chunks onward to the embedding model.
- 5. What data enters it?** Full plain text of one document, already extracted from its original format.
- 6. What processing does it perform?** Splits at ~512 tokens max, 0 overlap (splits at sentence/section boundaries instead), merges fragments under 256 tokens into a neighbor, additionally produces 150-token "mini-chunks," and can group into 4× larger "large chunks" for a coarser pass.
- 7. What does it output?** A list of text chunks tagged with document and position (for later citation).
- 8. Where does that output go?** To the embedding model — or first to a cheap LLM call for a context summary, if Contextual RAG is enabled (off by default here).
- 9. Why this approach?** Splitting on sentence/section boundaries (vs. blind fixed-character splits) preserves natural units of meaning, directly improving retrieval quality.
- 10. What if it stopped working?** No document could ever be indexed — everything downstream depends on having chunks in the first place.

REAL-WORLD ANALOGY

An editor splitting a huge reference book into index cards, careful never to cut a definition in half, so each card stands on its own.

13 Connectors (data source integrations)

- 1. What is it?** Code that knows how to talk to one specific external system — Slack, Google Drive, Confluence, a raw upload — and translate its content into one common internal shape.
- 2. Why does it exist?** Every external system has a different API, auth method, and data shape. Without this abstraction, every downstream step would need special-case logic per source.
- 3. When does it activate?** On a recurring schedule (Celery Beat checks for due syncs) or immediately on manual upload/trigger.
- 4. What communicates with it?** A Celery worker instantiates and calls it; it reaches the external system and returns results to the same worker.
- 5. What data enters it?** Encrypted credentials from PostgreSQL, plus a marker of what's new since the last sync.
- 6. What processing does it perform?** Authenticates, fetches raw content, and converts each item into a standard `Document` — text, title, URL, timestamps, and permission information carried over from the source.
- 7. What does it output?** A stream of standardized `Document` objects.
- 8. Where does that output go?** Into deduplication, then the chunker, then embedding, then OpenSearch — identical pipeline regardless of source.
- 9. Why this design?** A shared interface (load everything / poll for changes / list IDs for cleanup) lets each connector be built, tested, and fail independently. This deployment intentionally exposes only 5 of ~54 implemented connectors — a scope decision, not a technical limit.
- 10. What if it stopped working?** No new content enters from that source; existing indexed data stays searchable but silently goes stale.

REAL-WORLD ANALOGY

A customs officer/translator at one specific border crossing, fluent in exactly one country's paperwork, converting whatever arrives into a standard form before it proceeds inland.

14 MinIO (object storage)

- 1. What is it?** Self-hosted storage for raw files — an uploaded PDF's actual bytes — behaving like Amazon S3 but run on your own infrastructure.
- 2. Why does it exist?** PostgreSQL is built for small structured rows, not large binary blobs — storing a 50MB PDF there bloats the database and slows backups. Object storage is purpose-built for "store this file, give me a reference" at any size.
- 3. When does it activate?** On file upload, or when the system generates a downloadable artifact.
- 4. What communicates with it?** FastAPI writes on upload and reads to serve/reprocess; Celery workers read during document processing.
- 5. What data enters it?** Raw file bytes plus a retrieval key.
- 6. What processing does it perform?** Essentially none beyond reliably storing/retrieving bytes by key — deliberately simple.
- 7-8. Output & destination.** Raw bytes, returned to whichever component (typically a Celery worker) asked, which hands them to text extraction and the chunker.
- 9. Why this technology?** Keeps PostgreSQL small and fast while file storage scales independently; MinIO specifically is S3-API-compatible, so the same code works self-hosted or later migrated to real AWS S3 without rewrites.
- 10. What if it stopped working?** Uploads fail immediately; already-indexed text remains searchable since that lives in OpenSearch, not MinIO.

REAL-WORLD ANALOGY

The warehouse storing physical boxes, while PostgreSQL is the inventory ledger recording precise facts about each box without holding the box itself.

15 Docker (containers)

1. What is it? A way to package software with everything it needs to run into a self-contained "container" that behaves identically on any machine.

2. Why does it exist? Eliminates "it works on my machine" — a container ships the exact environment the software was built and tested in, every time.

3. When does it activate? Continuously — every component discussed above runs inside its own container, all the time.

4. What communicates with it? Not application data — Docker is the substrate everything else runs on. Docker Compose starts/stops/networks the containers so they find each other by name.

5-7. Input, processing, output. Takes a container image and a configuration describing networking/resources/env vars; isolates each container's filesystem/process/network while allowing controlled internal communication; outputs running, isolated processes.

9. Why this technology? Over installing directly on a server: consistency across dev, test, and production — the exact container tested locally runs in production. Scaling is as simple as starting more copies of a given container. Docker specifically is the dominant standard, maximizing compatibility with hosts and orchestration tools.

10. What if it stopped working? Nothing would be running — if the host machine crashed, the entire system is down until it restarts.

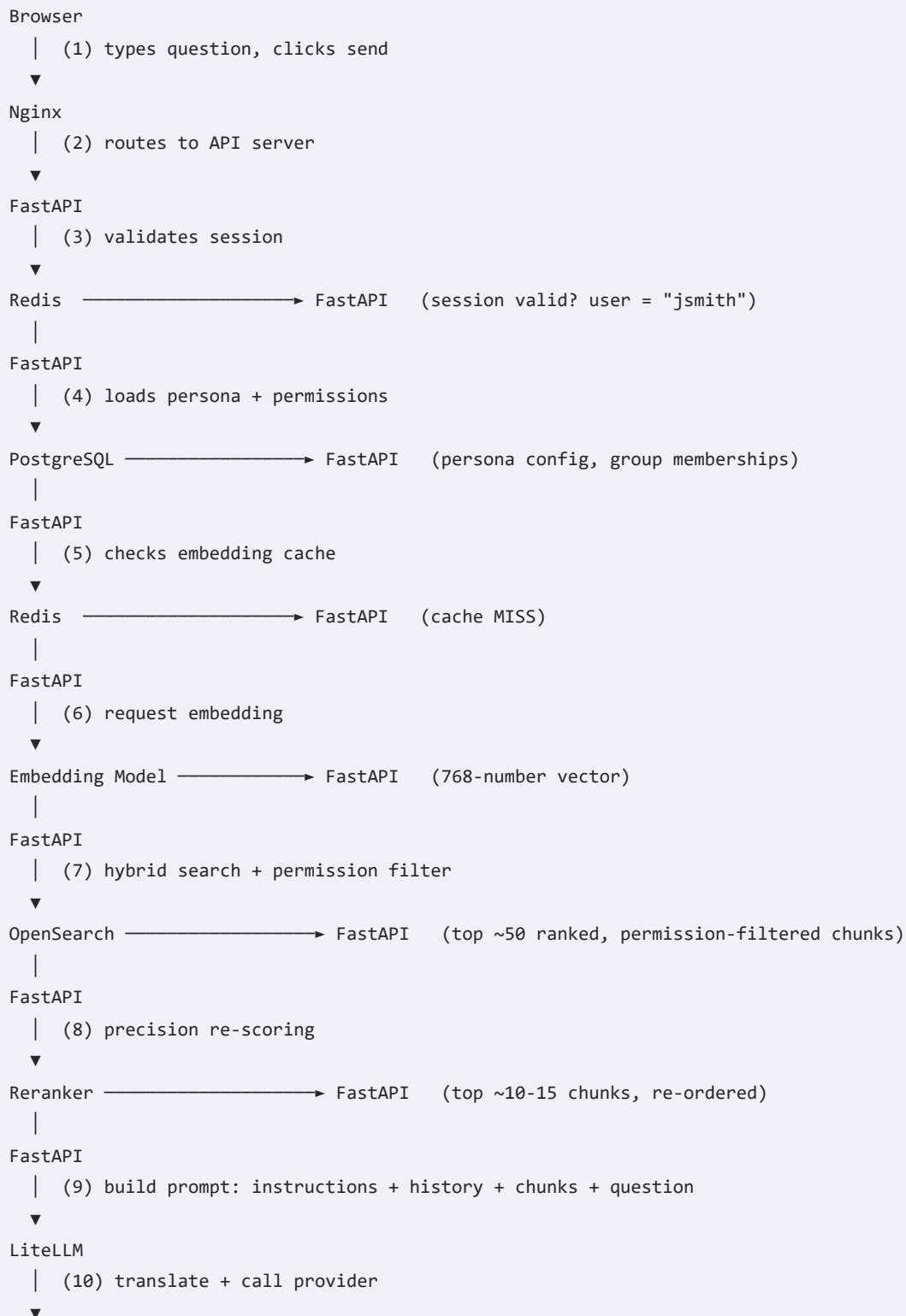
REAL-WORLD ANALOGY

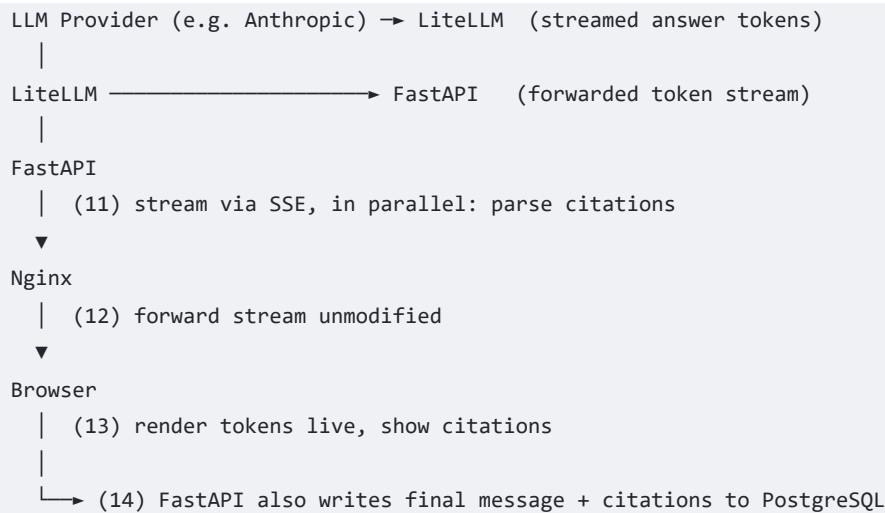
Standardized shipping containers in global freight — contents can be anything, but identical external dimensions mean any ship or crane can move them without knowing what's inside.

II.02 Story One — "What is our company's vacation policy?"

Every hop, click to
streamed answer

A logged-in user types this question and hits Enter. Every arrow below is a real, distinct step — what moves across it, why it can't be skipped, and what breaks if it vanished.





Step by step

Step 1 — Browser → Nginx. An HTTPS POST carries `{"message": "What is our company's vacation policy?", "chat_session_id": "..."}` plus the session cookie. The browser has no data or AI of its own — it can only forward intent. If this step vanished, the question would stay trapped on the user's own machine forever.

Step 2 — Nginx → FastAPI. The decrypted request is routed to the API container based on its path. Skipping this would mean exposing the raw application server directly to the internet — a serious security regression.

Step 3 — FastAPI → Redis (session check). The cookie value is checked; Redis answers "user jsmith" or "unknown/expired." Everything downstream depends on knowing who's asking. If Redis were down here, authentication fails for everyone.

Step 4 — FastAPI → PostgreSQL (persona + permissions). FastAPI loads the persona's scoped document sets and jsmith's group memberships. Skipping this means either searching everything indiscriminately (a scope violation) or not knowing who the user is (a security violation).

Step 5 — FastAPI → Redis (embedding cache). The question is hashed and checked; assume a **miss** — nobody asked this exact phrasing in the last 15 minutes. Skipping this cache entirely would still work correctly, just wastefully — recomputing identical vectors for repeated questions.

Step 6 — FastAPI → Embedding Model. The question text becomes a 768-number vector — the only thing that can be mathematically compared against stored chunk vectors. Without it, only exact keyword matches would be possible.

Step 7 — FastAPI → OpenSearch (hybrid search). The query vector, raw text, and jsmith's permission/scope info go in; the top ~50 chunks come back — already filtered to only what jsmith may see. This is the actual retrieval — the "R" in RAG. Without it, the LLM would answer from generic training knowledge only, ungrounded and with no permission enforcement.

Step 8 — FastAPI → Reranker. The question is paired individually with each of the ~50 candidates; the top 10-15 are kept, re-ordered — say, three chunks specifically about accrual and carryover rise to the top. Skipping this leaves OpenSearch's original ranking as-is, with slightly lower precision on close calls.

Step 9 — FastAPI builds the prompt. Persona instructions, recent history, the winning chunks (labeled by source), and the question are assembled into one block. An LLM has no memory and no built-in access to company data — everything it needs must be handed over explicitly, every time.

Step 10 — FastAPI → LiteLLM → LLM Provider. The prompt is translated and sent to the configured provider (say, Anthropic), which streams back tokens: "Our", " company", " provides", " 15", " days"... This is the actual language-generation step, turning retrieved facts into a coherent answer.

Step 11 — FastAPI → browser, via SSE. Each fragment is forwarded the instant it's received — not buffered — while FastAPI watches for citation markers and maps them to source chunks. Without SSE, the user would stare at a blank screen for the entire generation time instead of seeing words appear almost immediately.

Steps 12-13 — Nginx → Browser → rendering. The stream passes through unmodified; the browser's JavaScript appends each fragment live and renders citation links once that data arrives.

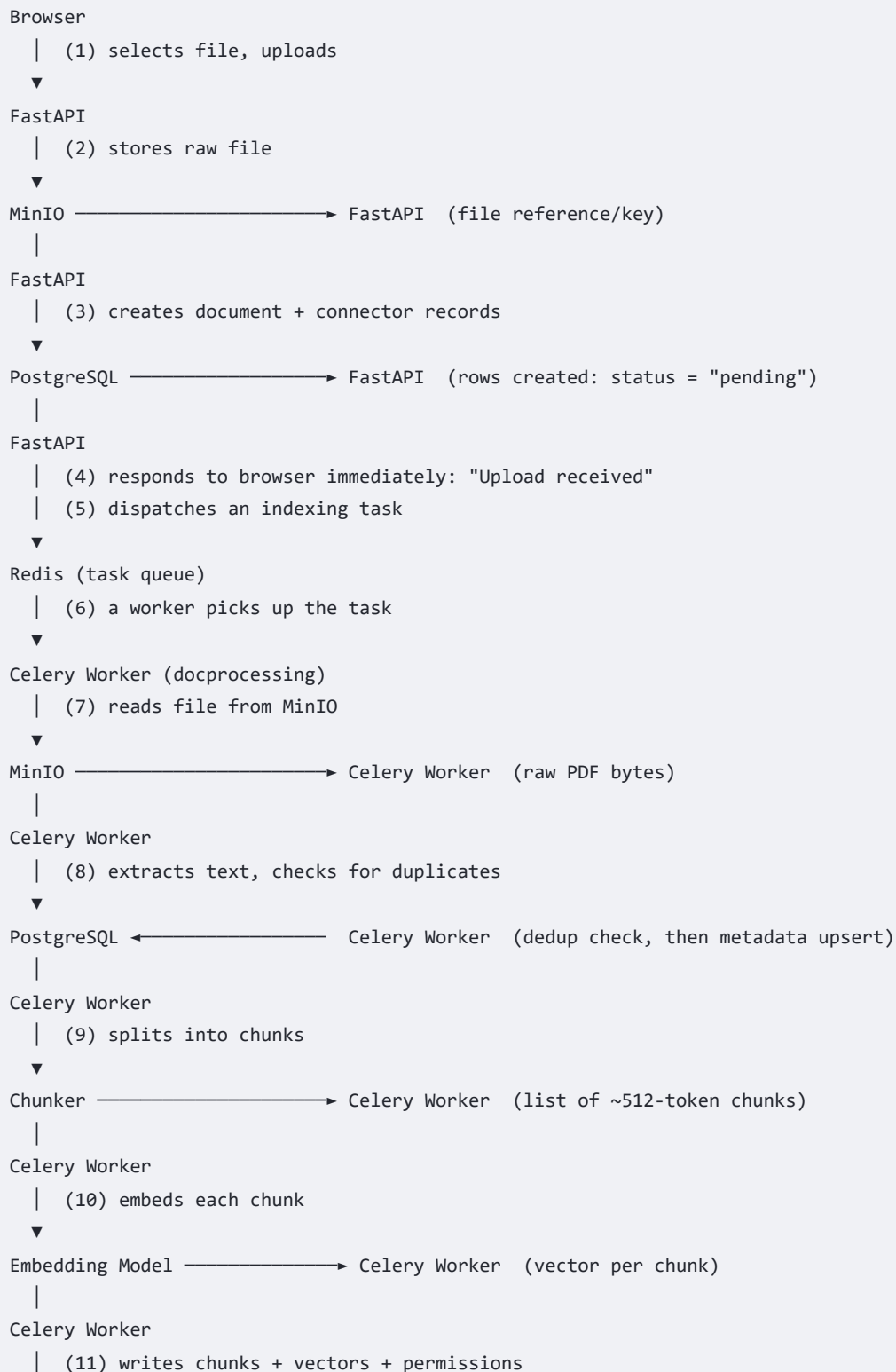
Step 14 — FastAPI → PostgreSQL (persistence, in parallel). The complete assistant message and citations are written to the `ChatMessage` table. Without this, the conversation would vanish the instant the tab closed.

That's the entire round trip — roughly a dozen distinct hops, each with one non-overlapping job, typically completed in a few seconds.

II.03 Story Two — A New PDF Is Uploaded

Every hop, click to searchable document

Someone clicks "Upload" and selects `Vacation-Policy-2026.pdf`. This flow is mostly asynchronous — the user gets an immediate response while the real work happens off to the side.



```
▼
OpenSearch
|
Celery Worker
| (12) marks job complete
▼
PostgreSQL (status = "success" — document is now searchable)
```

Step by step

Step 1 — Browser → FastAPI. Raw file bytes (multipart upload) plus target connector/permission scope. Obviously nothing can be indexed that was never received.

Step 2 — FastAPI → MinIO. The raw bytes are stored; MinIO returns a reference key for later retrieval. Large binary content doesn't belong in PostgreSQL.

Step 3 — FastAPI → PostgreSQL. A `Document` row (title, source = "file", MinIO reference) is created with status "pending" — this is how the rest of the system knows the document exists at all.

Step 4 — FastAPI → Browser (immediate response). "Upload received, processing." This is the entire reason background processing exists — waiting for full chunk/embed/index would freeze the tab and tie up server resources other users need.

Steps 5-6 — FastAPI → Redis → Celery Worker. A small task message is dropped into a queue; a `docprocessing` worker picks it up within moments — the actual handoff from live request to independent background job.

Step 7 — Celery Worker → MinIO. The worker fetches the raw bytes back out using the stored reference key — the PostgreSQL row only ever held a pointer.

Step 8 — Celery Worker ↔ PostgreSQL (extraction + dedup). After extracting readable text, the worker checks: has this exact content (by hash) already been indexed? Assume it's genuinely new. Without dedup, repeated uploads or connector re-syncs of unchanged content would waste compute and pollute the index with duplicate chunks.

Step 9 — Celery Worker → Chunker. The full extracted text becomes a list of ~512-token, boundary-aware chunks — without this, embedding would have nothing sensibly sized to work with.

Step 10 — Celery Worker → Embedding Model. Each chunk becomes a 768-number vector, in bulk, ahead of time, for content that will be searched later by someone not yet in the picture.

Step 11 — Celery Worker → OpenSearch. Chunk text, vector, and permission metadata (carried from the upload's configured scope) are written in. This is the only moment permission data attaches to a chunk — done wrong, either nobody could find the document later, or everybody could see it regardless of actual access rights.

Step 12 — Celery Worker → PostgreSQL (completion). Status flips from "pending" to "success." This is how the admin UI shows "Indexing Status: Complete" and prevents needless reprocessing of this exact version.

From this moment on, the document is fully searchable — the next time any authorized user asks a semantically or textually matching question, it will be found exactly like any pre-existing document, at Step 7 of Story One.

II.04 The System in Layers

Eight layers, and why each exists

Every component belongs to one of eight layers. A layer only depends on the layer(s) directly below it, never skipping ahead — which is what makes the system maintainable: Layer 5's internals (say, the LLM provider) can be completely replaced without Layer 1 ever changing.

Layer 1 — User Interface

Browser, Next.js

The only layer a human directly perceives. Turns application data into something understandable, and clicks back into structured requests. Depends on every layer below to have anything meaningful to show; nothing below depends on it.

Layer 2 — Reverse Proxy

Nginx

Thin traffic control between the outside world and everything internal — routing and encryption only, zero awareness of "documents" or "chat messages." Exists purely to protect and organize access to Layer 3.

Layer 3 — Backend / API

FastAPI

The decision-making core — every access rule and orchestration step lives here. Depends on Layer 4 for delegated slow work, Layer 5 for AI, Layer 6 for facts; Layers 1-2 depend entirely on it.

Layer 4 — Background Processing

Celery, Redis-as-broker

Exists purely to take slow work off Layer 3's plate. Depends on Layer 6 (facts) and Layer 5 (AI calls during processing); Layer 3 depends on it whenever work shouldn't block a live request.

Layer 5 — AI Layer

Embedding Model, Reranker, LiteLLM/LLM Providers

Every piece of actual machine intelligence — text-to-vector, relevance judgment, language generation. Called by Layers 3 and 4; never initiates anything itself.

Layer 6 — Databases

PostgreSQL

The permanent memory of who/what/when. Every layer above eventually reads or writes here; it depends on nothing above it.

Layer 7 — Search Engine

OpenSearch

A specialized sibling to Layer 6, dedicated to finding the most relevant chunks fast while respecting permissions. Depends only on already having received embedded chunks from Layer 5.

Layer 8 — Infrastructure

Docker, MinIO, Redis-as-cache

The physical foundation everything runs on. Every layer above ultimately depends on this being healthy — if the host goes down, all seven layers above disappear simultaneously.

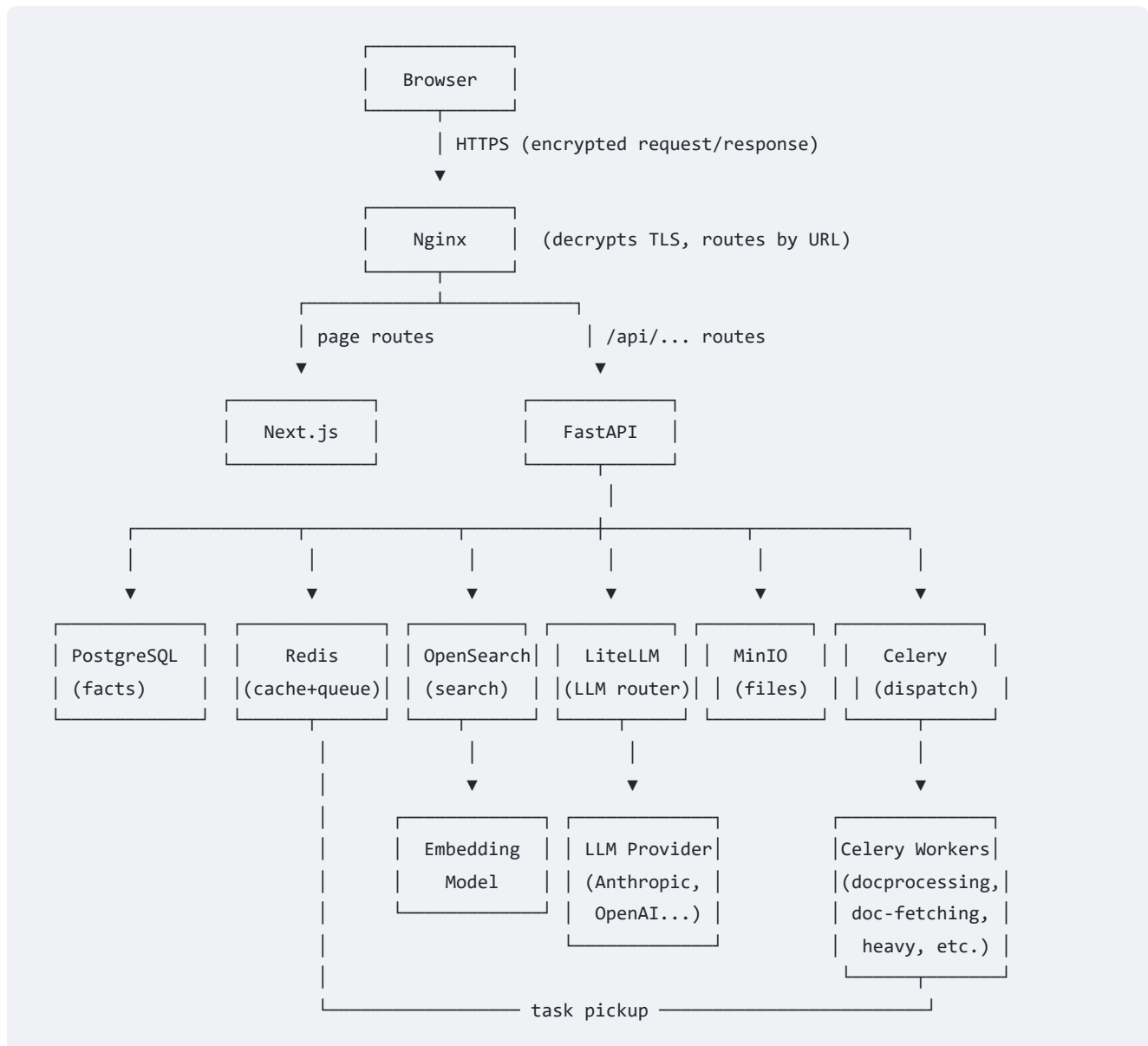
A HOSPITAL ANALOGY FOR THE WHOLE STACK

Layer 1 is the reception waiting room where a patient describes symptoms. Layer 2 is the triage desk directing them to the right department. Layer 3 is the attending physician making every real decision. Layer 4 is lab technicians running tests in the background while the doctor sees other patients. Layer 5 is specialist consultants (radiologists, pathologists) called in for expert judgment. Layer 6 is medical records — the permanent, trusted history. Layer 7 is a specialized research index the doctor searches for relevant literature fast. Layer 8 is the hospital building itself — power, water, and physical rooms that make everything else possible.

II.05 The Full System, Visually

Complete architecture, arrow by arrow

The complete static architecture — what's connected to what, at rest, regardless of any single request:



Arrow-by-arrow, in plain words

- **Browser** → **Nginx** — the encrypted request/response for everything the user does.
- **Nginx** → **Next.js** — requests for actual pages (HTML/JS/CSS).
- **Nginx** → **FastAPI** — requests for data/actions (`/api/...`).
- **FastAPI** → **PostgreSQL** — reads/writes of structured facts (users, chats, connectors, permissions).
- **FastAPI** → **Redis** — session checks, embedding-cache checks, and dropping new background task messages.
- **FastAPI** → **OpenSearch** — search queries during a chat, and (via Celery) new chunk writes.
- **FastAPI** → **LiteLLM** → **LLM Provider** — the prompt going out, and the streamed answer coming back.

- **FastAPI** → **MinIO** — storing an uploaded file, or retrieving one to serve/reprocess.
- **Redis** → **Celery Workers** — task pickup: a worker polling the queue grabs the next job.
- **Celery Workers** → **Embedding Model** — bulk text-to-vector conversion during indexing.
- **Celery Workers** → **OpenSearch** — writing newly chunked, embedded, permission-tagged content into the index.
- **Celery Workers** → **PostgreSQL** — updating a document's indexing status.

II.06 Quick-Reference Glossary

Every concept introduced, in one place

Term	One-line meaning
Reverse proxy	The single public front door that routes requests to the correct internal server.
TLS / HTTPS	Encryption between browser and server so nobody in between can read the traffic.
API	A defined contract for how one program asks another program to do something.
Session / cookie	A small token proving "this browser is logged in as this user," checked on every request.
Relational database	Stores structured, interlinked data (tables/rows) with strict correctness guarantees.
Cache	Temporary storage of an expensive result so it doesn't need to be recomputed.
TTL	How long a cached value is trusted before it's automatically discarded.
Message broker / task queue	A waiting line of background jobs, picked up by workers independent of live requests.
Worker	A running process whose job is to pull tasks off a queue and execute them.
Token	Roughly a word-fragment; the unit LLMs and embedding models actually process.
Embedding	A list of numbers representing the meaning of a piece of text, positioned in space.
Vector search	Finding stored items whose embedding is mathematically closest to a query's embedding.
BM25	A keyword-frequency scoring algorithm, good at exact term/proper-noun matches.
Hybrid search	Running vector search and BM25 together and merging the results.
Bi-encoder	A model that embeds text independently, fast, used for broad first-pass search.
Cross-encoder / reranker	A model that judges a (query, document) pair together, slow but precise.
Chunking	Splitting a long document into smaller, retrievable pieces before embedding.
RAG	Retrieval-Augmented Generation: retrieve real data first, then let the LLM write an answer grounded in it.
SSE (Server-Sent Events)	A one-way stream letting the server push an answer to the browser piece by piece.
Object storage	Storage purpose-built for raw files/blobs, separate from structured database rows.

Container (Docker)	A packaged, isolated, reproducible unit containing software plus everything it needs to run.
RBAC	Role-Based Access Control: permissions assigned via roles (Admin, Curator, etc.) rather than individually.

Where to go next. If you want to go one level deeper on any single piece — for example, exactly how the permission filter is implemented inside an OpenSearch query, or what happens if two users hit "send" on the same question at the same millisecond — that's a natural continuation of this document.